

# Phase-Based SDLC Agent Architecture

---

Redesigning Cursor AI configuration from flat skills  
to phase-based SDLC agents with structured handoffs

LKInsurance Project

April 2026

## Table of Contents

---

- 1. Architecture Overview
- 2. Directory Structure
- 3. Context Budget per Agent
- 4. Phase 1: Discovery Agent
- 5. Phase 2: Design Agent
- 6. Phase 3: Implementation Agent (Coordinator + Sub-Agents)
- 7. Phase 4: Testing Agent
- 8. Phase 5: Deployment Agent
- 9. Quick Fix Agent
- 10. What to Keep, Consolidate, or Remove
- 11. Artifact Templates
- 12. Migration Path

# 1. Architecture Overview

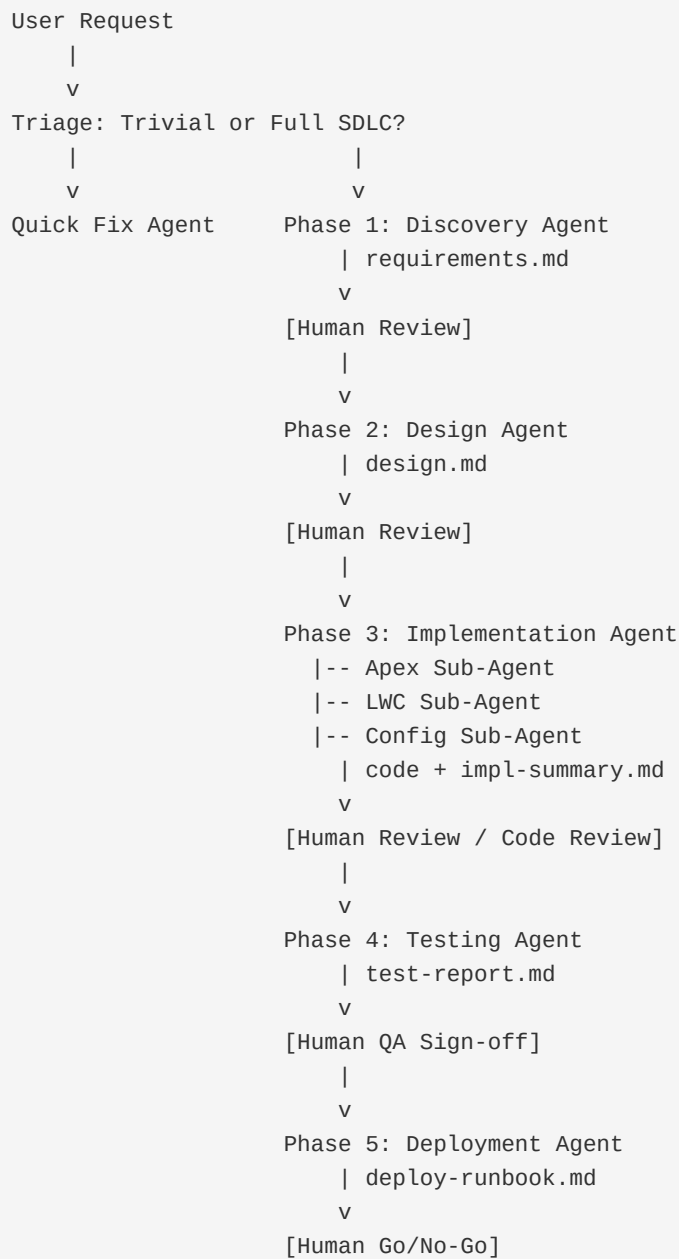
---

Replace the current flat skill + always-on rules model with a 6-agent system organized around SDLC phases. Each agent has a focused context, produces structured markdown artifacts, and requires human approval before the next phase begins.

Key principles:

- One agent per SDLC phase (focused context, no bloat)
- Structured markdown artifacts for handoff between phases
- Human review checkpoint between every phase
- Quick Fix agent for trivial tasks that bypass the full pipeline
- Sub-agents within Implementation for Apex, LWC, and Flow/Config

The pipeline flow:



## 2. Directory Structure

```

.cursor/
|-- rules/
|   |-- foundation/                # Layer 1: generic SF rules
|   |   |-- naming-conventions.md
|   |   |-- security-baseline.md
|   |   +-- governor-limits.md
|   |-- phase-1-discovery.md
|   |-- phase-2-design.md
|   |-- phase-3-implementation.md
|   |-- phase-4-testing.md
|   |-- phase-5-deployment.md
|   +-- quick-fix.md
|
|-- skills/
|   |-- discovery/SKILL.md          # Phase 1 agent
|   |-- design/SKILL.md            # Phase 2 agent
|   |-- implementation/SKILL.md     # Phase 3 coordinator
|   |   |-- apex/SKILL.md          # Apex sub-agent
|   |   |-- lwc/SKILL.md           # LWC sub-agent
|   |   +-- config/SKILL.md        # Flow/declarative
|   |-- testing/SKILL.md           # Phase 4 agent
|   |-- deployment/SKILL.md        # Phase 5 agent
|   +-- quick-fix/SKILL.md         # Quick Fix agent
|
|-- artifacts/
|   +-- templates/
|       |-- requirements.md
|       |-- design.md
|       |-- impl-summary.md
|       |-- test-report.md
|       +-- deploy-runbook.md
|
+-- tools/
    +-- generate-project-context.py

```

### 3. Context Budget per Agent

Each agent loads only the rules relevant to its phase, dramatically reducing context window usage compared to the current ~320KB+ per conversation.

| Agent          | Rules Loaded                                                     | Est. Context |
|----------------|------------------------------------------------------------------|--------------|
| Discovery      | foundation/ + phase-1-discovery.md + project context             | ~40KB        |
| Design         | foundation/ + phase-2-design.md + project ctx + requirements.md  | ~60KB        |
| Implementation | foundation/ + phase-3-impl.md + project ctx + req + design       | ~80KB        |
| Testing        | foundation/ + phase-4-testing.md + project ctx + prior artifacts | ~60KB        |
| Deployment     | foundation/ + phase-5-deploy.md + project ctx + prior artifacts  | ~40KB        |

|           |                                              |       |
|-----------|----------------------------------------------|-------|
| Quick Fix | foundation/ + quick-fix.md + project context | ~30KB |
|-----------|----------------------------------------------|-------|

## 4. Phase 1: Discovery Agent

**Purpose:** Understand the requirement, query the org, produce a structured requirements document.

### Rules Loaded

- foundation/ (naming, security, governor limits -- light reference)
- phase-1-discovery.md (business analysis patterns, requirement gathering, MCP org queries)
- Project context (07-active-project-context.md)

### Input

User requirement (natural language, Jira ticket, or markdown file)

### Process

1. Ask clarifying questions (functional, non-functional, constraints)
2. Query org via Salesforce MCP (existing objects, fields, flows, validation rules)
3. Identify impacted objects and existing automation
4. Assess: is this trivially solvable with configuration? (early exit to Quick Fix)

### Output Artifact: requirements.md

- Feature name, description, priority
- Functional requirements (numbered)
- Non-functional requirements (performance, security, accessibility)
- Impacted objects and fields (from MCP)
- Existing automation/config found (from MCP)
- Constraints and assumptions
- Acceptance criteria
- Open questions

**Human Checkpoint:** Review requirements, approve or revise.

## 5. Phase 2: Design Agent

**Purpose:** Produce a solution design based on approved requirements.

### Rules Loaded

- foundation/ (shared subset)

- phase-2-design.md (Well-Architected Framework, Configuration-First, architecture and data model patterns, integration patterns)
- Project context

## Input

Reads artifacts/{feature-name}/requirements.md

## Process

1. Evaluate Configuration-First (Flow vs Apex vs Validation Rule vs Formula)
2. If declarative: document the configuration approach
3. If code needed: design data model, component architecture, integration points
4. Assess against Well-Architected pillars (TRUSTED, EASY, ADAPTABLE)
5. Identify what sub-agents are needed in Implementation (Apex? LWC? Flow?)

## Output Artifact: design.md

- Approach decision (declarative vs code, with rationale)
- Data model changes (new objects/fields, ERD)
- Component architecture (Apex classes, LWC components, Flows)
- Integration points (if any)
- Security considerations
- Well-Architected assessment
- Implementation sub-agents needed (Apex, LWC, Config)
- Risk assessment

**Human Checkpoint:** Review design, approve architecture.

# 6. Phase 3: Implementation Agent

---

**Purpose:** Build the solution per approved design. Coordinator dispatches to sub-agents.

## Coordinator

- Reads requirements.md + design.md
- Determines which sub-agents to invoke based on the 'Implementation sub-agents needed' section from design.md
- Dispatches work in the right order (e.g., Apex service layer before LWC that calls it)
- Validates integration between sub-agent outputs

## Apex Sub-Agent

**Rules:** foundation/ + phase-3-implementation.md + Apex-specific patterns (trigger framework,

bulkification, service layer, CRUD/FLS)

**Produces:** Apex classes and triggers with proper naming, security, and bulkification

## LWC Sub-Agent

**Rules:** foundation/ + phase-3-implementation.md + LWC-specific patterns (component structure, @wire, events, SLDS, accessibility)

**Produces:** LWC components with proper structure, SLDS styling, and accessibility compliance

## Config Sub-Agent

**Rules:** foundation/ + phase-3-implementation.md + declarative patterns (Flows, Validation Rules, Formula Fields, Approval Processes)

**Produces:** Flow definitions, validation rule specs, configuration instructions

### Output Artifact: impl-summary.md

- Files created/modified (with paths)
- Apex classes and their responsibilities
- LWC components and their responsibilities
- Flows/config changes
- Known limitations
- Dependencies between components

**Human Checkpoint:** Code review.

## 7. Phase 4: Testing Agent

---

**Purpose:** Produce test classes, test plans, and validate coverage.

### Rules Loaded

- foundation/ (shared)
- phase-4-testing.md (testing standards, 75%+ coverage, bulk testing, mock callouts, Jest patterns, test data factories)
- Project context

### Input

Reads all prior artifacts + actual code from Phase 3

### Process

1. Generate Apex test classes (bulk-safe, 200+ records, test data factory)



2. Generate Jest tests for LWC components
3. Create test plan (positive, negative, bulk, edge cases, security)
4. Validate test coverage targets

#### **Output Artifact: test-report.md**

- Test classes created (with paths)
- Test plan (scenarios, expected results)
- Coverage targets per class
- Edge cases tested
- Bulk test scenarios
- Security test scenarios

**Human Checkpoint:** QA sign-off.

## **8. Phase 5: Deployment Agent**

---

**Purpose:** Prepare deployment package and runbook.

#### **Rules Loaded**

- foundation/ (shared)
- phase-5-deployment.md (CI/CD, package.xml, change sets, scratch org, validation, rollback)
- Project context

#### **Input**

Reads all prior artifacts

#### **Output Artifact: deploy-runbook.md**

- Deployment method (source deploy, change set, package)
- Pre-deployment steps
- Deployment commands
- Post-deployment validation
- Rollback plan
- Smoke test checklist

**Human Checkpoint:** Go/no-go decision.

## **9. Quick Fix Agent**

---

**Purpose:** Handle trivial tasks that don't warrant 5 phases.

**Rules Loaded:** foundation/ + quick-fix.md (light rules covering naming, security, basic testing)

## Scope

- Typo fixes, simple renames
- Single field additions
- Simple validation rule changes
- Label/translation updates
- Bug fixes with obvious root cause

**Guardrail:** If the Quick Fix agent determines the task is non-trivial (touches multiple objects, needs architecture decisions, has security implications), it must redirect to Phase 1: Discovery instead.

## 10. What to Keep, Consolidate, or Remove

---

### Keep

- generate-project-context.py and Layer 3.5 (project context switching) -- works well, each phase agent reads it
- NotebookLM integration pattern (but centralize IDs into one config file)
- Salesforce MCP integration (used primarily by Discovery and Design agents)

### Consolidate

- Merge the 8 duplicated rule files (root 00-03 + layer-1-universal 00-03) into a single foundation/ set
- Fix the SOQL limit error (150 -> 100) during consolidation
- Merge Layer 4 methodology rules into the relevant phase rules (SPSM stages map to phases, Configuration-First goes into Design, Plan-First is now structural)

### Remove

- astro/SKILL.md -- replaced by the phase pipeline structure itself (no orchestrator needed when the workflow is the orchestrator)
- staffing-manager/SKILL.md -- team personas are no longer needed; each phase agent has its own identity
- All 14 hardcoded persona names and warm introductions
- Duplicated rule files

### Repurpose

Existing skill content from fullstack-dev, solution-architect, lwc-developer, qa-engineer, etc. gets redistributed into the appropriate phase agent rules:

- Architecture patterns -> Design agent
- Apex patterns -> Implementation / Apex sub-agent
- LWC patterns -> Implementation / LWC sub-agent

- Testing patterns -> Testing agent
- Integration patterns -> Design agent + Implementation coordinator

## 11. Artifact Templates

Each template is a structured markdown file with clear sections. Agents fill in the sections; humans review them. The next phase agent reads the completed artifact.

**Key design principle:** Artifacts should be self-contained -- the Design agent should be able to understand the feature from requirements.md alone, without needing the Discovery conversation history.

Artifact summary:

| Phase          | Artifact          | Key Sections                                                               |
|----------------|-------------------|----------------------------------------------------------------------------|
| Discovery      | requirements.md   | Functional reqs, NFRs, impacted objects, acceptance criteria               |
| Design         | design.md         | Approach decision, data model, component arch, Well-Architected assessment |
| Implementation | impl-summary.md   | Files created, class responsibilities, component map, dependencies         |
| Testing        | test-report.md    | Test classes, test plan, coverage targets, edge cases                      |
| Deployment     | deploy-runbook.md | Deploy method, commands, rollback plan, smoke tests                        |

## 12. Migration Path

Recommended order of execution:

1. Consolidate rules first -- fix duplications, contradictions (SOQL limit 150->100), merge into foundation/
2. Create artifact templates -- define the handoff contract between phases
3. Build phase agents one at a time -- Discovery first, then Design, Implementation, Testing, Deployment
4. Redistribute existing skill content -- move architecture patterns to Design, Apex patterns to Implementation sub-agent, testing patterns to Testing agent, etc.
5. Build Quick Fix agent last -- simplest agent, uses consolidated foundation rules
6. Test end-to-end -- run a real feature through all 5 phases to validate the handoff artifacts and human review checkpoints

---

End of Document